SHARED EXECUTION GRAPHS: A SUBSTRATE FOR MULTI-AGENT AND HUMAN-AI COLLABORATION

Josh Rosen
ThruWire, Inc.

Seth Rosen
ThruWire, Inc.

May 6, 2026

ABSTRACT

Large language model systems increasingly perform work that is collaborative, multi-step, and persistent across time. Yet most human-AI and multi-agent systems still coordinate through loops: one chat per user, one transcript per agent, and one growing pile of notes, memory entries, documents, tickets, or messages used to reconstruct state at update time. Those mechanisms preserve communication, but they are usually weak as execution substrates. They do not ordinarily preserve execution identity, explicit dependency semantics, artifact invalidation, or reliable downstream propagation when upstream reasoning changes.

This paper argues that collaboration for AI-native work should be grounded in a shared execution graph rather than isolated chat loops or shared memory alone. A shared execution graph is a durable directed acyclic graph of blocks, tasks, or notebooks with explicit dependencies, replayable execution identities, inspectable intermediate artifacts, and incremental recomputation semantics. It is shared because multiple humans, agents, and tools can read, edit, execute, review, and consume the same graph state. In this model, collaboration is not merely communication about work. It is co-authoring and executing a shared computational structure.

We position this paper as the collaboration-model counterpart to the first two papers in this series. If Paper 1 asked how AI-native work should execute and Paper 2 asked what durable state that execution should produce, this paper asks how multiple participants should coordinate over that executable state. We distinguish shared execution from shared memory, compare graph-based collaboration to chat rooms, shared documents, wikis, git repositories, task queues, knowledge graphs, vector memory, blackboard systems, and message-passing multi-agent loops, and argue that these alternatives usually lack the dependency-aware propagation semantics required for maintained collaborative state.

The contribution is a systems-level abstraction, not a claim that shared graphs eliminate conflict or replace all collaboration interfaces. Graph authoring has overhead, many tasks remain well served by lightweight conversation, and conflict resolution still requires policy and sometimes human judgment. The narrower claim is that for long-lived AI-native work, a shared execution graph is a stronger collaboration substrate than shared chat, shared memory, or shared documents because it preserves explicit dependency structure, execution identity, and consistent propagation under change.

1 Introduction

Human-AI collaboration is often described as conversation. A user asks, the model answers, follow-up questions accumulate, and the thread grows into a working session. Multi-agent systems generalize the same pattern: one agent plans, another researches, another critiques, and coordination happens through message passing, role prompts, shared notes, or a memory store [1, 3, 4, 5]. This pattern is productive because it matches the interface of current language models. The simplest control loop is also the default interface.

That interface, however, becomes structurally weak once work is long-lived. Many real tasks are not single conversational episodes. They produce intermediate analyses, assumptions, plans, decision artifacts, transformations, and revisions that must remain coherent across repeated edits by multiple humans and multiple agents. In these settings, the system's main challenge is no longer only answer generation. It is maintaining shared state under change.

Papers 1 and 2 in this series argued that long-lived AI-native work should be modeled as explicit execution graphs that produce durable intermediate artifacts. Paper 1 introduced *execution lineage* as the runtime substrate for deterministic replay and selective recomputation. Paper 2 introduced intermediate artifacts as first-class state, with lineage, supersession, and maintained-state quality as central concerns. Those arguments imply a third question that becomes unavoidable once more than one actor is involved:

> If AI-native work executes as a graph and produces durable artifact state, how should multiple humans and agents collaborate over that state?

The common answer today is some mixture of messaging and auxiliary stores. Agents talk to each other. Humans leave instructions in chats, documents, tickets, wikis, or pull requests. Systems maintain long-term memory in vector stores or knowledge bases. These mechanisms are useful, but they usually coordinate *around* the work rather than *through* the work. They preserve facts, notes, messages, or outputs, yet they often do not preserve which exact execution produced an artifact, which downstream artifacts depend on it, whether an upstream change invalidates that artifact, or what must be recomputed next.

This distinction motivates the paper's core claim:

> Collaboration is not merely communication; it is co-authoring and executing a shared computational structure.

We call that structure a *shared execution graph*. It is a directed acyclic graph (DAG) of blocks, notebooks, or tasks with explicit dependency edges, replayable execution identities, typed intermediate artifacts, and propagation semantics. It is shared because all participants—humans, autonomous agents, code assistants, tools, and external services—can inspect and mutate the same substrate. Messages may still exist, but they are no longer the authoritative state boundary. The graph is.

This shift matters because *shared memory is not the same as shared execution*. A memory store may tell an agent that a recommendation changed. A shared execution graph tells the system which block changed, which artifact it superseded, which downstream blocks now have stale reads, which branches remain unaffected, and which recomputations or reviews should follow. The first is a coordination hint; the second is a collaboration substrate.

The paper makes four contributions. First, it defines the shared execution graph as the collaboration-model extension of execution lineage and artifact lineage. Second, it formalizes actors, mutations, execution identity, and propagation semantics for multi-human and multi-agent work. Third, it introduces a concurrency and conflict model centered on graph mutation, stale state detection, invalidation, attribution, and reviewable edits rather than on informal message coordination alone. Fourth, it proposes an evaluation framework in which collaborative systems are judged not only by final output quality, but also by maintained collaborative state quality over time.

The scope is intentionally restrained. We do not claim that all collaboration should become graph-native, that conversation is obsolete, or that shared graphs remove disagreement. Many tasks do not justify graph authoring overhead. Some collaborative ambiguity is irreducibly social. Agents can also make poor structural edits, and poorly designed dependency semantics can create false confidence. The narrower thesis is that when work is multi-step, multi-actor, revisable, and execution-dependent, a shared execution graph is a materially stronger substrate than shared chat transcripts, message loops, or generic memory stores.

2 Motivating Example

Consider a strategy workflow shared by two humans and three agents. A human researcher authors an upstream market-structure block. A research agent expands the graph with competitor profiles and customer segmentation artifacts. A planning agent consumes those artifacts to draft rollout options. A finance-minded human edits an upstream assumptions block to require that year-one launch remain budget-neutral. A review agent checks whether the downstream strategy memo and implementation plan still conform.

In a loop-centric system, this work commonly fragments. The human researcher has one chat. The planning agent has another. The review agent reads a shared document plus a few notes in a wiki or ticket system. The updated budget constraint may be mentioned in a message, copied into a planning brief, and partially reflected in a revised

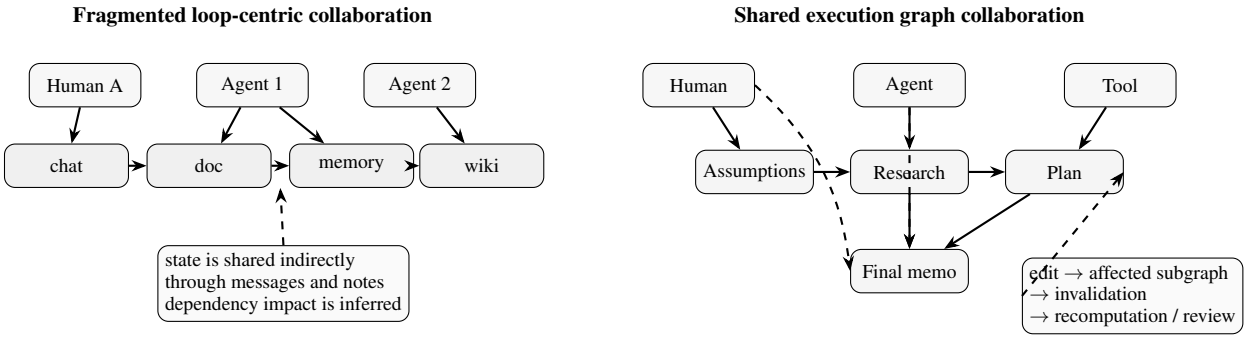

Figure 1: Messages and shared memory can coordinate participants, but they usually lack explicit execution and propagation semantics. A shared execution graph gives humans, agents, and tools one authoritative substrate for edits, replay, invalidation, and recomputation.

memo. But there is no authoritative computational object that knows exactly which downstream artifacts depend on the changed assumption. One agent may continue using the old segmentation summary. Another may rewrite the final memo correctly while leaving the rollout plan stale. A human reviewer can eventually reconcile these inconsistencies, but the system itself does not know where the inconsistency boundary is.

Now contrast a shared execution graph. The market assumptions, competitor profiles, segmentation, rollout criteria, implementation plan, and final memo are explicit blocks with explicit dependencies. The budget-neutrality change is a graph mutation against a concrete upstream block or artifact. The runtime can mark downstream artifacts as stale, preserve unrelated branches exactly, and schedule recomputation or review where appropriate. Humans and agents can both inspect the affected subgraph before choosing whether to recompute automatically, request approval, or manually edit a downstream node.

The difference is not merely better storage. Both systems may preserve the same text somewhere. The difference is that one preserves *execution structure*. In the graph case, participants share the same object of collaboration: a dependency-aware, executable, replayable structure. In the loop case, collaboration is mediated by communication plus reconstruction.

Figure 1 illustrates the contrast. On the left, state fragments across chats, documents, and memory stores, with downstream impact inferred socially or heuristically. On the right, humans, agents, and tools all operate on a shared execution graph whose explicit dependencies support invalidation and propagation.

3 Background and Related Work

3.1 Agent Loops and Multi-Agent Coordination

ReAct [1] made the reasoning-and-acting loop the canonical LLM-agent pattern. Toolformer [2] showed how tool use can be integrated into model behavior. Multi-agent systems such as CAMEL [3], AutoGen [4], MetaGPT [5], and Voyager [6] extend this idea through role specialization, conversation protocols, and iterative critique. These systems demonstrate that complex tasks can be decomposed across interacting agents even when the dominant coordination primitive remains messaging.

The important limitation for our purposes is not that message passing fails. It often works. The limitation is that coordination state is commonly encoded in transcripts, agent-local memory, or controller logic rather than in a shared execution substrate. Even when several agents work on one task, their notion of "current state" often depends on prompt reconstruction rather than on a common dependency-aware object.

3.2 Execution Lineage and Artifact-Centric State

Paper 1 in this series argued that AI-native work should execute through explicit DAG structure with replayable execution identities, deterministic forward execution, and selective recomputation boundaries. Paper 2 argued that such execution requires durable intermediate artifacts with lineage, addressability, and supersession semantics. The present paper inherits those abstractions directly. A shared execution graph is therefore not merely a task graph. It is an execution-lineage graph whose nodes publish artifact-centric state.

This framing is adjacent to recent work on agent harnesses and workflow memory [7, 8, 9, 10, 11, 12, 13]. Harness research recognizes that important behavior lives outside the raw model invocation. Memory research recognizes that long-lived agents require durable external state. Our claim is narrower and more structural: memory helps an actor remember; a shared execution graph defines the collaboration substrate over which actors coordinate, mutate, and propagate work.

3.3 Workflow DAGs, Provenance, and Replay

Workflow systems such as Dryad [14], Spark [15], Airflow [16], and dbt [17] long ago converged on explicit dependency graphs, materialized intermediates, and lineage-aware recomputation. Provenance research similarly studies how outputs can be traced to prior state [18, 19]. The relevance is not metaphorical. These systems show that when work becomes multi-stage and collaborative, implicit dependencies and ad hoc handoffs become bottlenecks for correctness and maintainability.

LLM workflows differ because node-local computation may be stochastic and semantically rich. But the need for explicit dependency semantics becomes stronger, not weaker, when collaboration involves revisable intermediate reasoning rather than only deterministic transforms.

3.4 Blackboard Systems, Shared Memory, and Collaborative Editing

Blackboard architectures are a historically important comparison because they also provide a shared space where multiple knowledge sources contribute partial work. Nii's classic blackboard formulation emphasized a common store visible to specialist modules [20]. That tradition correctly recognized that shared intermediate state can outperform isolated agent behavior. Our distinction is that a generic blackboard usually does not by itself define replayable execution identity, explicit downstream invalidation semantics, or graph-scoped recomputation boundaries for AI-native work. A shared execution graph can be seen as a more execution-precise substrate built around lineage and dependency structure rather than only shared posting.

Collaborative editing research, including operational transformation and CRDTs [21, 22], is also relevant because it studies how concurrent edits can remain intelligible and mergeable. We do not claim that graph collaboration reduces to text-collaboration algorithms. The point is more limited: shared execution graphs inherit the need for explicit concurrency semantics, stale-read handling, attribution, and conflict visibility. Those questions are familiar in CSCW and distributed systems even if the objects being edited are now executable blocks and artifacts rather than plain documents.

3.5 Human-AI Collaboration

Human-AI interaction research has increasingly emphasized oversight, review, trust, and interface structure rather than pure model accuracy [23]. That perspective aligns with our evaluation stance. The relevant system quality is not only whether the model can produce a good answer once, but whether humans can understand what changed, what remains stale, what needs review, and how the system is maintaining shared state over time.

4 Problem Formulation

We consider AI-native work that is collaborative along at least one of three axes:

- multiple humans contribute edits, review, or decisions
- multiple agents produce, transform, or critique intermediate work
- work persists across time and must be revised under changing assumptions

The dominant loop-centric approach typically treats each actor as owning a local context window and coordinating through messages or externalized notes. Let $A = \{a_1, \ldots, a_n\}$ be actors, each with local state L_i, and let coordination occur through messages M plus an auxiliary shared store S:

$$\text{collaboration}_{\text{loop}} = \{L_i\}_{i=1}^{n} + M + S \tag{1}$$

This arrangement can preserve a large amount of information, but it leaves four recurrent problems under-specified.

Execution identity is weak. When an output is revised, the system often cannot say which exact prior execution it corresponds to, whether it is a replay of prior work, or whether it is merely a fresh generation that resembles earlier content.

Dependency structure is weak. Messages and documents may mention upstream relationships, but they usually do not make those relationships operational. The system often does not know which downstream work is affected when upstream reasoning changes.

Propagation is heuristic. If an upstream assumption changes, propagation is commonly manual, prompt-assembled, or partial. Systems may rewrite the final answer while leaving stale intermediates elsewhere.

Conflict visibility is weak. Two actors can make incompatible edits against overlapping state without the system surfacing that incompatibility until late human review, because the authoritative collaboration object is ambiguous.

These failures are all variants of the same structural issue: *shared state is not the same as shared execution*. A wiki page, vector store, or task queue can preserve shared information while remaining silent about which computations that information should invalidate or regenerate.

We therefore define the target object differently. Let

$$G = (V, E) \tag{2}$$

be a directed acyclic graph of executable blocks or tasks, where each node $v \in V$ publishes artifacts and each edge $(u, v) \in E$ is an explicit dependency relation. We want collaboration to occur through mutations of G and of its artifact state, not only through messages about G.

The systems question is then:

> What collaboration semantics are gained when humans, agents, and tools share one executable dependency graph with stable execution identities and propagation rules?

5 Shared Execution Graph Model

5.1 Definition

A *shared execution graph* is a tuple

$$\mathcal{C} = (G, X, R, \Pi) \tag{3}$$

where:

- $G = (V, E)$ is an executable DAG of blocks or tasks
- X is the current set of published artifacts and execution records
- R is the set of actors authorized to inspect, mutate, execute, or review parts of the graph
- Π is the policy layer governing invalidation, recomputation, approval, and conflict handling

Each node v has a structural specification σ_v, a resolved input surface x_v, a local execution procedure f_v, and a published output boundary o_v. Following Paper 1, we define execution identity as:

$$k_v = h(\sigma_v, x_v, \{k_u : u \in \mathrm{pred}(v)\}) \tag{4}$$

where $\mathrm{pred}(v)$ are the immediate predecessors of v. Following Paper 2, the published result of a node is not just text but a durable artifact surface with lineage and status.

The collaboration-specific extension is that multiple actors may inspect or mutate σ_v, x_v, dependency edges, publication metadata, or review status over time. Those edits operate against explicit graph structure rather than only against informal narrative context.

5.2 Why Shared Execution Is Stronger Than Shared Memory

A shared memory system provides a common place to read and write information. A shared execution graph provides a common place to read and write *dependency-bearing computational state*. The difference matters in three ways.

Identity. Graph nodes and published artifacts have identities grounded in executions and dependencies, not only in document names or memory keys.

Affectedness. The graph can enumerate affected downstream work after an edit. A generic shared memory store usually cannot do this without an added dependency model.

Propagation. The graph can invalidate, replay, or recompute downstream work consistently. A shared memory store may preserve the changed fact but not the semantics of what to do next.

This distinction is the collaboration analogue of the earlier distinction between prompt lineage and execution lineage. Shared memory preserves recall; shared execution preserves coordinated maintenance.

5.3 Graph Mutations as Collaboration Units

In loop-centric collaboration, the primitive unit is often the message: "please update the memo," "agent 2 found a new source," or "assume budget neutrality." In shared execution collaboration, the stronger primitive is the graph mutation. A mutation can:

- add a node or dependency edge
- edit a node specification or prompt contract
- supersede an artifact
- trigger invalidation or recomputation
- mark a node for review, repair, or approval

Messages can still accompany these operations, but the authoritative state change is the mutation against the graph or its published artifacts. This makes collaboration semantics stronger than conversational intent alone.

6 Actors, Mutations, and Execution Identity

6.1 Actors

We distinguish four broad actor classes.

Humans. Humans may author structure, edit assumptions, inspect intermediate artifacts, approve or reject changes, resolve conflicts, and interpret ambiguous goals.

Autonomous agents. Agents may propose new nodes, execute existing nodes, critique downstream outputs, monitor stale regions, repair failed nodes, or synthesize summaries over graph state.

External tools and services. Tools may populate source artifacts, run validators, render outputs, execute code, or provide external data used by graph nodes.

Interactive clients. Chat clients, code assistants, or MCP-driven interfaces may present graph state conversationally while still reading from and writing to the graph as the authoritative backend.

The important point is that these actors can differ in interface while sharing the same substrate. One actor may experience the system as a visual DAG, another as a notebook-like builder, another as a chat interface with tool access. The collaboration model does not require one UI. It requires one authoritative execution graph.

6.2 Mutations

Let a mutation be an operation m that transforms collaboration state:

$$m : \mathcal{C}_t \to \mathcal{C}_{t+1} \tag{5}$$

Examples include:

- editing upstream assumptions
- adding a new analysis branch
- changing a dependency edge
- superseding an intermediate artifact
- re-running a stale block
- attaching a review decision to a recomputed node

Mutations matter because they are the place where collaboration becomes operational. A message saying "budget neutrality now matters" is not yet a maintained-state operation. A mutation that updates a criteria block or supersedes a recommendation artifact is.

6.3 Execution Identity as Collaboration Grounding

Execution identity provides the collaboration substrate with a concrete referent. Actors need not speak only about vague conversational state such as "the analysis you did earlier." They can refer to a specific block execution or artifact version. This enables:

- exact replay rather than approximate regeneration
- review of the precise state a downstream node consumed
- auditability of who changed what and against which prior execution
- reasoning about stale reads and recomputation scope

This is especially important in multi-agent work. Without execution identity, two agents can appear to operate on the "same" upstream note while actually consuming different prompt reconstructions or artifact versions. The graph eliminates some of this ambiguity by grounding collaboration in explicit execution records.

6.4 Extending the Graph

An important difference between graph-centric collaboration and question-answer chat is that agents can extend the graph itself. They are not limited to emitting free-form answers. An agent can add a competitor-analysis node, create a downstream risk review branch, or propose a repair block for inconsistent artifacts. Humans can approve these extensions, reject them, or rewire their dependencies.

This matters because collaborative AI work is often not just about filling in one output. It is about creating new structure as understanding evolves. A shared execution graph makes that structural growth inspectable and reviewable.

7 Consistency, Invalidation, and Propagation

7.1 Consistency as a Graph Property

In collaborative AI systems, consistency should not be framed only as agreement among messages. It should be framed as a property of the maintained graph state. A collaborative state is more coherent when:

- active artifacts agree with the current upstream assumptions they depend on
- superseded artifacts are not mistaken for current ones
- unaffected branches remain stable under irrelevant edits
- affected downstream branches are identifiable and reviewable

Paper 2 described maintained-state quality at the artifact level. Shared execution graphs generalize that concern to multiple actors maintaining a common dependency structure over time.

7.2 Invalidation Before Recomputation

An upstream edit should first create an invalidation judgment and only then a recomputation judgment. This sequencing matters. Without an explicit invalidation phase, downstream consumers can continue reading stale artifacts as if they were current while recomputation is pending or awaiting approval.

Formally, if node u changes so that its execution identity becomes $k'_u \neq k_u$, then every reachable descendant v whose identity function depends on k_u becomes stale with respect to current state:

$$\text{stale}(v) = 1 \quad \text{if} \quad k_v \text{ was computed from } k_u \text{ and } k'_u \neq k_u \tag{6}$$

The system may then choose among replay, recomputation, manual review, or temporary suspension based on policy Π. But stale status should be visible before those downstream actions complete.

7.3 Propagation Semantics

Shared execution graphs support three useful propagation outcomes.

Exact preservation. If an edit occurs on an unrelated branch, unaffected downstream artifacts should remain exactly preserved. This is the no-op locality result already emphasized in Paper 1's evaluation.

Selective recomputation. If an upstream artifact changes in a dependency-relevant way, only affected descendants should be invalidated and recomputed.

Superseding publication. When recomputation produces a new current artifact, downstream consumers should see the superseding artifact as authoritative while historical versions remain available for audit.

These propagation semantics are stronger than what shared documents or chats normally provide. Those systems can record that something changed; they usually do not operationalize what the change should invalidate.

7.4 Inspectability

Propagation is useful only if participants can inspect it. Humans and agents should be able to ask:

- what changed?
- which upstream execution or artifact changed?
- which downstream nodes are now stale?
- which artifacts were preserved exactly?
- which outputs were replayed, recomputed, or manually edited?

This is a major advantage over transcript-mediated collaboration. In a large chat or wiki-based workflow, these questions often require manual reconstruction from narrative history.

8 Concurrency and Conflict Semantics

8.1 Why Concurrency Needs Its Own Model

Once multiple humans and agents can edit a shared graph, concurrency stops being an implementation detail. It becomes part of the collaboration model. Two agents may propose edits to the same block. A human may revise upstream assumptions while an agent is recomputing downstream artifacts. A review agent may consume an artifact that was current when review began but has become stale before review completes.

We do not claim that shared execution graphs eliminate such conflicts. The claim is narrower: they make conflicts *more legible* because they occur against explicit structure rather than hidden conversational state.

8.2 Graph Mutation as the Unit of Collaboration

The natural concurrency unit is the graph mutation, not the raw chat turn. This has several consequences.

Optimistic edits. Actors can propose mutations optimistically against the latest visible graph state.

Validation at publish time. Before a mutation becomes authoritative, the system can validate whether the targeted block, artifact, or dependency surface is still current.

Conflict localization. Conflicts can be scoped to overlapping nodes, artifacts, or edges rather than treated as generic conversational divergence.

8.3 Conflicts on the Same Block

If two actors edit the same block specification, prompt contract, or authoritative artifact, the graph should surface this as a direct structural conflict. Depending on policy, the system may:

- require human resolution before publication
- maintain both variants as explicit branches
- apply deterministic merge rules for non-overlapping fields
- let one actor's mutation supersede another under a priority policy

The point is not that one universal policy is correct. The point is that the shared graph gives the system a concrete place to detect and represent the conflict. In chat-centric collaboration, such conflicts may remain hidden until their consequences appear in downstream prose.

8.4 Stale Reads and Stale Artifacts

A separate class of conflict is the stale read. An actor may inspect artifact a_t while another actor has already published superseding artifact a_{t+1}. The collaboration substrate should therefore distinguish:

- current artifacts
- historical but still inspectable artifacts
- stale downstream artifacts awaiting recomputation or review

This is one reason invalidation should precede recomputation. It lets the system mark the state of dependencies explicitly rather than silently letting downstream actors proceed with outdated assumptions.

8.5 Review and Approval Workflows

Some graph mutations can execute automatically; others should trigger review. For example:

- an agent may auto-recompute a low-risk formatting block
- a changed decision-criteria artifact may require human approval before its descendants are republished
- a repair agent may propose a structural dependency change that must be reviewed before execution

The graph is useful here because approval applies to explicit objects: a mutation, a node, an artifact supersession, or a recomputation set. The review surface is therefore more precise than "please check this latest chat output."

8.6 Actor Attribution and Audit Logs

Collaborative graph state should record attribution for authored structure, published artifacts, invalidation decisions, recomputations, and approvals. Attribution improves:

- auditability
- debugging
- responsibility assignment
- trust calibration for downstream consumers

This is especially important in mixed human-agent settings. A human reviewer should be able to determine whether a stale branch persists because it was never recomputed, because an agent repair failed, or because a human intentionally overrode automatic propagation.

8.7 Relation to OT and CRDT-Style Thinking

Operational transformation and CRDTs study how concurrent edits can converge in collaborative systems [21, 22]. Shared execution graphs inherit part of that problem but with different objects. The main challenge is not only merging text. It is preserving coherent dependency semantics under concurrent mutations. Some subproblems may admit OT- or CRDT-like treatment, especially for block-local text or metadata fields. But the harder collaboration questions concern execution affectedness, stale reads, approval policy, and whether two concurrent edits imply branching or invalidation. Those are semantic workflow questions, not only data-structure questions.

9 System Design

9.1 Research Abstraction from the Sibling Repository

The sibling ThruWire repository provides a concrete implementation context from which a more general research abstraction can be extracted. Its architecture separates authored project state, compilation, runtime execution, persistence, and tools access. Projects are authored as blocks with dependencies, compiled into normalized graph structure, and executed by a runtime that treats execution as a materialization pass over a known DAG. Runtime state includes frames, scratchpads, dependency entries, artifact entries, and step execution records. Blocks execute concurrently when dependencies allow, while steps remain sequential within a block.

For this paper, the important lesson is not any product-specific UI. It is that collaboration can be grounded in a shared project graph whose execution and persistence layers already distinguish:

- project state versus run state
- raw authored blocks versus compiled graph structure
- dependency entries versus local artifact entries
- selected final artifacts versus narration or control events

This is precisely the kind of separation a collaboration substrate needs.

9.2 Blocks, Steps, and Local Typed State

The sibling runtime treats a block execution as owning one frame, one shared scratchpad, one ordered step sequence, and one local execution history. This is a useful collaboration boundary. It implies that local step interaction can remain flexible and even loop-like, while publication to shared state occurs at deliberate boundaries. In other words, a shared execution graph does not ban inner loops. It places them inside explicit structural containers.

That design supports a hybrid view of collaboration:

- within a block, an agent may reason iteratively
- across blocks, collaboration is governed by explicit dependencies and replayable publication boundaries

This is a stronger structure than exposing all collaboration as one giant transcript.

9.3 Project Execution and Regeneration

Another practical lesson from the sibling repository is the separation between project revisions and run replay. Project-service style storage maintains editable latest state and compiled snapshots, while the runtime layer stores replayable step surfaces, block-cache lookups, and execution indexes. Abstracted into research terms, this means collaboration happens over two coupled but distinct objects:

- the editable shared graph definition
- the replayable execution state materialized from that graph

This distinction is valuable because collaboration often edits structure and results at different times. A human may edit the graph without immediately recomputing every descendant. An agent may regenerate one stale region while another branch remains pending approval. Keeping project structure and execution state distinct but linked makes these workflows legible.

9.4 Shared Editing Without Product Lock-In

The same abstraction is compatible with multiple interfaces. A visual editor may expose blocks and dependencies directly. A chat assistant may convert conversational requests into proposed graph mutations. A code-centric client may edit block definitions in files. An MCP tool may execute, diff, or inspect graph state programmatically. The substrate remains the same if all of these clients read from and write to the same authoritative graph and execution records.

This is why the paper is not a product paper. The substantive claim is not about one interface. It is about the stronger semantics of a shared executable graph as the underlying collaboration object.

10 Evaluation

10.1 Evaluation Objective

The right question is not simply whether a collaboration system produces a good final answer once. It is whether the shared collaborative state remains coherent over time. We therefore argue that collaborative AI systems should be evaluated on both:

- final output quality
- maintained-state quality under collaborative change

This follows directly from Papers 1 and 2. Final answer quality alone can hide stale or contradictory intermediate state. In multi-actor settings, that problem becomes worse because several contributors may build on different local reconstructions of current reality.

10.2 Compared Conditions

Three comparison points are especially useful.

Single shared chat or transcript. Humans and agents coordinate in one or more conversation threads, perhaps with attached notes or pasted context, but without explicit dependency semantics.

Separate agent loops with shared memory or wiki state. Each agent has its own loop and local context, but shared notes, memory tools, or wiki pages provide common information.

Shared execution graph. All actors read from and write to one dependency-aware executable graph with replay, invalidation, and artifact supersession semantics.

The distinction between the second and third conditions is central. Shared memory may improve recall, but it does not by itself define which outputs are stale or what recomputation should follow an upstream change.

10.3 Implemented In-Repo Tasks

The repository already contains controlled telehealth-policy update tasks and metrics in the execution-lineage experiment harness. Those tasks are not multi-human multiplayer benchmarks by themselves, but they directly operationalize the propagation semantics Paper 3 depends on.

Unrelated-branch update. An isolated provider-recruiting branch changes while the final telehealth recommendation should remain untouched. This tests exact preservation, unaffected-artifact preservation, and contamination avoidance.

Intermediate-artifact edit. An upstream recommendation-criteria artifact is revised to require budget neutrality and utilization controls before expansion. This tests downstream propagation recall, artifact preservation, cross-artifact consistency, and recomputation locality.

These tasks already distinguish loop baselines from an artifact-first DAG harness. The metrics include stable artifact-hash preservation, unaffected-artifact preservation, downstream propagation recall, upstream churn rate, no-op churn, contamination rate, constraint reflection, and cross-artifact consistency. Those metrics are directly relevant to collaboration because they measure whether maintained state remains coherent under change rather than only whether the latest memo looks plausible.

10.4 Observed Repository Results

The existing in-repo results align with the central claim of this paper. In the unrelated-branch update, the DAG condition preserves the final memo exactly and avoids unrelated-branch contamination, while loop conditions often regenerate the memo and sometimes import staffing content. In the intermediate-artifact edit, all conditions can reflect the new constraint in the final memo, but the DAG condition more cleanly preserves unaffected upstream artifacts and maintains downstream consistency across the criteria artifact, implementation plan, and final memo.

These results should be interpreted carefully. They do not prove that shared execution graphs always produce better prose. They show something more structural and more relevant to collaboration: final output quality alone undermeasures coherence of maintained state. A loop can write a good revised memo while still leaving the surrounding shared state in partial disagreement. That is precisely the problem a collaboration substrate should address.

10.5 Proposed Multiplayer Evaluation Extension

To evaluate Paper 3 directly, the same harness logic can be extended to collaborative tasks such as:

- collaborative strategy authoring where one human edits assumptions while agents maintain downstream options and memo text
- research synthesis where multiple agents own different evidence branches and a human revises the decision criterion midstream
- multi-step planning where one upstream block changes after some descendants have already been reviewed
- maintenance workflows where a review agent monitors stale descendants and a repair agent recomputes or patches them

Useful metrics include:

- number of inconsistent downstream artifacts after an upstream edit
- time or steps required to identify the affected subgraph
- edit localization, measured as unnecessary unaffected-state churn
- propagation completeness, measured as recall over expected affected descendants
- duplicate work across actors
- human reviewer burden
- auditability and traceability of why state changed
- final output quality plus maintained-state quality

The central evaluation claim is therefore:

> For collaborative AI systems, final output quality alone is insufficient; the decisive question is whether shared state remains coherent, attributable, and correctly propagated over time.

11 Discussion

11.1 Collaboration Through Structure, Not Only Messaging

Messages remain useful. People and agents still need to ask questions, justify edits, negotiate goals, and summarize outcomes. The paper does not argue against messaging. It argues against treating messaging as the authoritative computational substrate. The stronger abstraction is to let messages explain or request changes, while the graph records the actual collaborative state.

This mirrors the earlier distinction between logs and program structure. Messages are valuable explanatory surfaces. They are weak execution boundaries.

11.2 Why Existing Shared-State Systems Are Insufficient

The comparison to existing systems can now be stated more precisely.

Chat rooms and transcripts. They preserve communication, not dependency-aware execution state.

Shared documents and wikis. They preserve text and discussion, but usually not execution identity, invalidation semantics, or automatic downstream affectedness.

Git repositories. They are powerful for code versioning and review, but they generally version files and patches rather than maintaining execution-aware dependency semantics for LLM-native intermediate artifacts.

Task queues. They can schedule work, but they typically do not preserve durable intermediate reasoning state or automatically know which outputs become stale when upstream reasoning changes.

Knowledge graphs and vector memory. They preserve facts, relations, or retrievable context, but usually not replay identity, publication authority, or recomputation semantics for collaborative AI workflows [24].

Blackboard systems and message-passing agents. They move closer by giving multiple specialists shared state or shared communication. But unless that state also carries explicit graph structure, artifact supersession, and downstream invalidation semantics, shared posting remains weaker than shared execution.

These systems are not useless. Many collaborative stacks will continue to use several of them. The paper's narrower claim is that they are insufficient as the sole authoritative substrate for long-lived AI-native work.

11.3 Multiplayer AI Work as Executable Co-Authoring

Once collaboration is grounded in a shared execution graph, multi-agent work looks less like a swarm of independent chats and more like executable co-authoring. Humans can adjust structure, approve supersessions, or repair assumptions. Agents can extend the graph, maintain stale branches, or monitor invariant violations. Tools can validate or render results. All of these actions become mutations against one shared computational object.

This is a stronger collaboration model because it aligns social coordination with computational structure.

12 Limitations

The argument has several important limits.

- **Graph authoring overhead.** Not every task benefits from explicit graph structure. Lightweight conversation is often cheaper and sufficient.
- **Structural complexity.** As graphs grow, they can become their own coordination burden. Explicit structure improves legibility only if it remains understandable.
- **Conflict policy is still required.** Shared graphs make conflicts clearer; they do not determine the correct resolution policy.
- **Agents can make poor structural edits.** A bad dependency edge or inappropriate artifact supersession can propagate mistakes more systematically.
- **Propagation quality depends on dependency correctness.** If the graph omits a real dependency or preserves a wrong artifact contract, invalidation and recomputation decisions will also be wrong.
- **Not all collaboration needs executable semantics.** Brainstorming, casual exploration, and loosely coupled discussion may not justify the discipline of a graph substrate.
- **Evaluation remains early.** The current in-repo tasks measure propagation and maintained-state quality in controlled settings, not full real-world multiplayer collaboration at organizational scale.

These limitations matter because the paper is not proposing a universal replacement for existing collaboration tools. It is proposing a stronger systems abstraction for one class of work: long-lived, AI-native, dependency-bearing collaboration.

13 Conclusion

Paper 1 in this series asked how AI-native work should execute and answered with execution lineage over deterministic graphs. Paper 2 asked what durable state that execution should produce and answered with first-class intermediate artifacts and artifact lineage. Paper 3 asks how multiple humans, agents, and tools should coordinate over that executable state.

The answer argued here is the shared execution graph. A shared execution graph is a stronger collaboration substrate than isolated chats, shared memory, or shared documents because it gives participants a common authoritative structure with explicit dependencies, execution identity, artifact supersession, invalidation semantics, and selective propagation under change.

The paper's central claim is deliberately narrow. Shared execution graphs do not solve collaboration, eliminate conflict, or replace conversation. But for long-lived AI-native work, they make collaboration more operationally precise. Participants can coordinate over concrete executions and artifacts rather than vague conversational state. Upstream edits can invalidate identifiable downstream work. Recomputations can be localized. Review can attach to specific mutations. Maintained shared state can be evaluated for coherence over time.

In that sense, the shared execution graph is not merely another memory layer. It is the executable substrate on which multiplayer AI work can become durable, inspectable, and maintainable.

References

[1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Thomas Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*, 2023.

[2] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, 2023.

[3] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. arXiv:2303.17760, 2023.

[4] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155, 2023.

[5] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. arXiv:2308.00352, 2023.

[6] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291, 2023.

[7] Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. Natural-Language Agent Harnesses. arXiv:2603.25723, 2026.

[8] Yuxuan Zhang, Haoyang Yu, Lanxiang Hu, Haojian Jin, and Hao Zhang. General Modular Harness for LLM Agents in Multi-Turn Gaming Environments. arXiv:2507.11633, 2025.

[9] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent Workflow Memory. arXiv:2409.07429, 2024.

[10] Shengda Fan, Xin Cong, Yuepeng Fu, Zhong Zhang, Shuyan Zhang, Yuanwei Liu, Yesai Wu, Yankai Lin, Zhiyuan Liu, and Maosong Sun. WorkflowLLM: Enhancing Workflow Orchestration Capability of Large Language Models. arXiv:2411.05451, 2024.

[11] Dongge Han, Camille Couturier, Daniel Madrigal Diaz, Xuchao Zhang, Victor Rühle, and Saravan Rajmohan. LEGOMem: Modular Procedural Memory for Multi-agent LLM Systems for Workflow Automation. arXiv:2510.04851, 2025.

[12] Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Hinrich Schütze, Volker Tresp, and Yunpu Ma. Memory-R1: Enhancing Large Language Model Agents to Manage and Utilize Memories via Reinforcement Learning. arXiv:2508.19828, 2025.

[13] Luiz C. Borro, Luiz A. B. Macarini, Gordon Tindall, Michael Montero, and Adam B. Struck. Memori: A Persistent Memory Layer for Efficient, Context-Aware LLM Agents. arXiv:2603.19935, 2026.

[14] Michael Isard, Mihai Budiu, Yuan Yu, Andrew Birrell, and Dennis Fetterly. Dryad: Distributed Data-Parallel Programs from Sequential Building Blocks. In *EuroSys*, 2007.

[15] Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. Spark: Cluster Computing with Working Sets. In *USENIX HotCloud*, 2010.

[16] Apache Software Foundation. Airflow Documentation: DAGs. `https://airflow.apache.org/docs/apache-airflow/3.0.4/core-concepts/dags.html`, accessed May 6, 2026.

[17] dbt Labs. dbt Developer Hub. `https://docs.getdbt.com/`, accessed May 6, 2026.

[18] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and Where: A Characterization of Data Provenance. In *International Conference on Database Theory*, 2001.

[19] Juliana Freire, David Koop, Emanuele Santos, and Claudio T. Silva. Provenance for Computational Tasks: A Survey. *Computing in Science & Engineering*, 10(3):11–21, 2008.

[20] H. Penny Nii. Blackboard Systems, Part One: The Blackboard Model of Problem Solving and the Evolution of Blackboard Architectures. *AI Magazine*, 7(2):38–53, 1986.

[21] Clarence A. Ellis and Simon J. Gibbs. Concurrency Control in Groupware Systems. In *Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data*, pages 399–407, 1989.

[22] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. A Comprehensive Study of Convergent and Commutative Replicated Data Types. INRIA Research Report RR-7506, 2011.

[23] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. Guidelines for Human-AI Interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, pages 1–13, 2019.

[24] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, Jos Lehmann, Roberto Navigli, Axel Polleres, and others. Knowledge Graphs. *ACM Computing Surveys*, 54(4):1–37, 2021.